

Usage Guide

This guide covers how to **use** the graph-db-engine tools: the command-line interface (`graphdb-cli`), the interactive agent REPL, and the terminal UI (`graphdb-tui`). For **examples** of specific capabilities (OCR, RAG, image generation, etc.), see [EXAMPLES.md](#). For the **Python SDK** API reference, see [python/SDK.md](#).

Table of Contents

1. [Quick Start](#)
 2. [CLI Reference \(`graphdb-cli` \)](#)
 - [Common Flags](#)
 - [Subcommands](#)
 3. [Interactive Agent REPL \(`agent` \)](#)
 - [Launch](#)
 - [Slash Commands](#)
 - [Skills](#)
 - [Tool-Call Trace](#)
 - [Ingesting Your Own Code](#)
 4. [Terminal UI \(`graphdb-tui` \)](#)
 5. [Model Selection](#)
 - [Default \(Offline Stub\)](#)
 - [What is `tokenizer.json` ?](#)
 - [Real Local GGUF Model](#)
 - [Where to Get Models](#)
 - [GPU Acceleration](#)
 6. [Configuration \(`settings.json` \)](#)
 - [Discovery Order & Precedence](#)
 - [Full Reference](#)
 - [Common Recipes](#)
 7. [Customizing & Extending](#)
 - [Adding a New Skill](#)
 - [Adding a New Tool](#)
 - [Training a Needle Model for a Task](#)
 8. [Engine Selection \(LLM vs. Needle\)](#)
 9. [Python SDK](#)
-

Quick Start

```
# Build the workspace (Rust crates: rust, model-hub, cli, tui)
cargo build --workspace

# Run the interactive agent (Claude-Code-style REPL)
cargo run -p graphdb-cli -- agent

# Ask a one-off question
cargo run -p graphdb-cli -- ask "what is the Needle engine"

# Launch the terminal UI (full-screen chat + RAG + extraction modes)
cargo run -p graphdb-tui

# Generate text from a prompt
cargo run -p graphdb-cli -- generate "Explain Rust ownership in one sentence"

# List all CLI subcommands
cargo run -p graphdb-cli -- --help
```

CLI Reference (`graphdb-cli`)

The `graphdb-cli` provides a command-line interface to all model-hub pipelines. Every subcommand runs with **offline stub models** by default (no weights, no network). Point `--model / --tokenizer` at real GGUF weights to run a quantized Llama instead.

Common Flags

These flags apply to **all** subcommands (place them **before** the subcommand name):

Flag	Description	Default
<code>--model <PATH></code>	Path to a GGUF text model file	None (uses offline stub)
<code>--tokenizer <PATH></code>	Path to the matching tokenizer.json	None (required if <code>--model</code> is set)
<code>--device <DEVICE></code>	Compute device: <code>cpu</code> , <code>cuda</code> , <code>metal</code>	<code>cpu</code>
<code>--settings <FILE></code>	Path to a <code>settings.json</code> (used by the <code>agent</code> sub-command)	auto-discovered (details)

Example:

```
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  --device cpu \
  generate "Explain Rust traits"
```

Subcommands

generate

Generate text from a prompt.

```
cargo run -p graphdb-cli -- generate "Write a haiku about Rust"
```

Flags:

- `--max-tokens <N>` — maximum new tokens (default: 256)
- `--temperature <T>` — sampling temperature, 0 = greedy (default: 0.7)

embed

Produce a text embedding vector.

```
cargo run -p graphdb-cli -- embed "the Rust programming language" --engine needle
```

Flags:

- `--engine <ENGINE>` — `llm` or `needle` (default: `needle`)

Prints the embedding dimension and a preview of the first 8 values.

rag

Retrieval-augmented generation over a small demo knowledge base.

```
cargo run -p graphdb-cli -- rag "what is Rust" --k 4 --engine needle
```

Flags:

- `--k <N>` — number of chunks to retrieve (default: 4)
- `--engine <ENGINE>` — embedding engine for re-ranking: `llm` or `needle` (default: `needle`)

Prints the final answer and the source chunks (file path + relevance score).

extract

Extract structured JSON from text according to a schema.

```
cargo run -p graphdb-cli -- extract "Alice works at OpenAI since 2021" \
  --schema entities --engine needle
```

Flags:

- `--schema <SCHEMA>` — a built-in name (`entities`) or an inline JSON schema string (default: `entities`)
- `--engine <ENGINE>` — extraction engine: `llm` or `needle` (default: `needle`)

Prints the extracted JSON.

ask

Run the agentic orchestrator (tool calling + answer composition) on the demo knowledge base.

```
cargo run -p graphdb-cli -- ask "list all documents" --engine needle
```

Flags:

- `--engine <ENGINE>` — tool-routing engine: `llm` or `needle` (default: `needle`)

Prints the selected tool and the final answer.

transcribe

Transcribe a mono f32 PCM raw audio file (stub unless a real Whisper model is loaded).

```
cargo run -p graphdb-cli -- transcribe /path/to/audio.raw
```

Expects little-endian f32 samples at 16 kHz.

agent

Launch the **interactive Claude-Code-style REPL**. See the [dedicated section below](#).

```
cargo run -p graphdb-cli -- agent
```

Flags:

- `--context-dir <DIR>` — auto-ingest `.rs` / `.md` / `.txt` files from this directory at startup
 - `--settings <FILE>` — path to a `settings.json` (global flag; see [Configuration](#))
-

info

Print system and build information: version, device, text model backend, GPU features, registered tools.

```
cargo run -p graphdb-cli -- info
```

Interactive Agent REPL (agent)

The `agent` subcommand launches a **Claude-Code-style REPL** — an interactive terminal session where you can ask questions about a codebase grounded in the graph knowledge base. It features:

- **Diamond prompt (◆)** with readline editing (arrow keys, history, Ctrl-C/D)
- **Difficulty routing** (Simple / Moderate / Complex) — each task is classified and routed to the appropriate engine(s)
- **Skill selection** — reusable instruction bundles loaded from disk (`model-hub/examples/skills/*/SKILL.md`)
- **Visible tool-call traces** — shows `● tool_use` and `⌌ tool_result` for every turn
- **Slash commands** — `/help`, `/skills`, `/tools`, `/context`, `/ingest`, `/clear`, `/quit`

Launch

```
# Fully offline (stub LLM):
cargo run -p graphdb-cli -- agent

# Ingest your own source tree at startup:
cargo run -p graphdb-cli -- agent --context-dir ./model-hub/src

# With a real local GGUF model:
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  agent

# Using a settings.json (no repeated flags – see the Configuration section):
cargo run -p graphdb-cli -- --settings ./graphdb-agent.settings.json agent
```

Configure once, run simply. Instead of passing `--model` / `--tokenizer` / `--context-dir` every time, put them in a `settings.json` and just run `cargo run -p graphdb-cli -- agent`.

Startup Banner

```
graphdb agent • Claude-Code-style REPL 

model      stub-text
skills     api-comparator, code-explainer, codebase-navigator, refactor-planner
tools      search_knowledge_base, list_documents, get_document_chunks
context    5 documents / 5 chunks

Type /help for commands, /quit to exit.

◆
```

Slash Commands

Type any of these at the `◆` prompt:

Command	Effect
<code>/help</code>	Show the full command table + routing table
<code>/settings</code>	Show the effective configuration and where it was loaded from
<code>/skills</code>	List loaded skills (name, description, trigger cues)
<code>/tools</code>	List available tools with their parameters
<code>/context</code>	Show how many documents and chunks are in the KB
<code>/ingest <path> [title]</code>	Read a file and add it to the knowledge base (<code>.rs</code> , <code>.md</code> , <code>.txt</code>)
<code>/clear</code>	Clear the terminal screen
<code>/quit</code> <code>/exit</code> <code>/q</code>	Exit the REPL

Skills

Skills are reusable instruction bundles stored as `model-hub/examples/skills/<name>/SKILL.md` . Each skill has:

- A **name** (e.g. `code-explainer`)
- A **description** (one-line summary)
- **Trigger cues** (comma-separated keywords like `"explain, how, why, describe"`)
- **Instructions** (the body of the `SKILL.md` , injected into the LLM prompt when the skill fires)

When you enter a task, the agent:

1. Classifies the difficulty (Simple / Moderate / Complex)
2. Selects the best-matching skill by counting cue hits
3. Injects that skill's instructions into the LLM prompt (if LLM composition is used)

Four skills ship by default:

Skill	Description	Trigger Cues
codebase-navigator	Locate files, list documents, fetch chunks	list, show, find, where, which file, locate, chunk, sections of, what documents
code-explainer	Explain how a piece of the codebase works	explain, how, why, what does, walk me through, describe, understand
refactor-planner	Plan an implementation or refactor	refactor, implement, design, add, change, migrate, re-write, introduce, build
api-comparator	Compare options or summarize trade-offs	compare, versus, vs, trade-off, tradeoff, difference, summarize, pros and cons

To **add a new skill**, drop a `SKILL.md` into `model-hub/examples/skills/<name>/` — no code changes needed. Format:

```
---
name: my-skill
description: One-line summary of what the skill does.
cues: keyword1, keyword2, keyword3
---
The instruction body that will be injected into the LLM prompt.
Keep it concise and directive (e.g., "Respond with a numbered list...").
```

Tool-Call Trace

Every turn prints a **visible trace** of the tool-calling loop (just like Claude Code):

```
◆ explain how the Orchestrator delegates work

→ COMPLEX reasoning/generation cue
+ skill code-explainer

● tool_use search_knowledge_base({"query":"explain how..."})
  ↳ tool_result [{"chunk":{"id":"/model-hub/src/pipeline/orchestrator.rs#0",...}}]
(120-char preview)

handled by stub-text route → search_knowledge_base → stub-text compose

[The Orchestrator is the main agent turn driver. It first selects a tool
via the configured engine (LLM or Needle), executes it against the graph,
then has the text model compose a final answer grounded in the tool result.
Engine selection is per-capability via PipelineConfig.]
```

Routing table:

Difficulty	Tool routed by	Answer composed by	Skill injected?
SIMPLE	Needle (deterministic)	Deterministic format (no LLM)	No
MODERATE	Needle	Local LLM	Yes
COMPLEX	Local LLM	Local LLM	Yes

Ingesting Your Own Code

At startup:

```
cargo run -p graphdb-cli -- agent --context-dir ./my-project/src
```

Every `.rs`, `.md`, and `.txt` file under `./my-project/src` (non-recursive) is ingested into the knowledge base before the REPL starts.

During a session:

```
◆ /ingest ./my-project/src/main.rs MyProject
✓ ingested MyProject → 12 chunks total
```

Now your own code is searchable and can be retrieved via the `search_knowledge_base` tool.

Persistent History

Command history is saved to `~/.graphdb_agent_history` and restored on the next run. Arrow keys (↑/↓) navigate it.

Terminal UI (`graphdb-tui`)

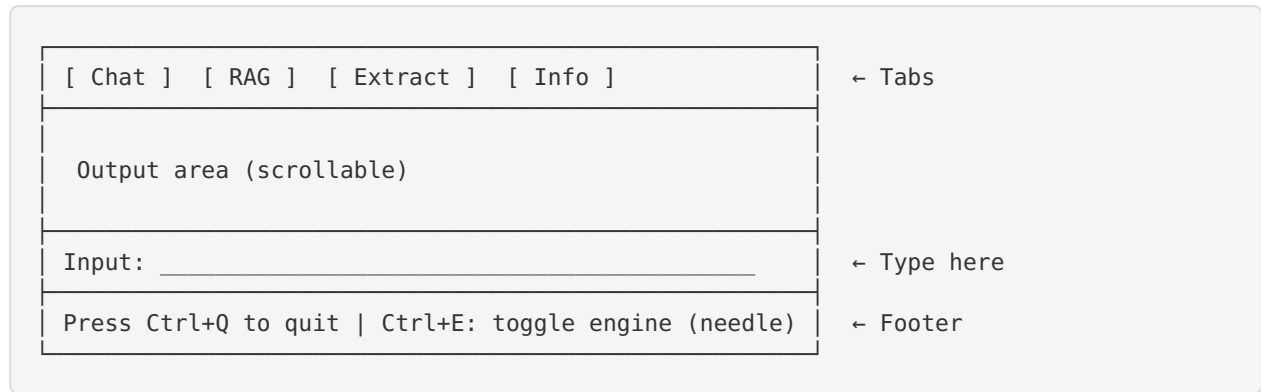
The `graphdb-tui` is a full-screen terminal UI for the model-hub pipelines, built with `ratatui` + `crossterm`. It provides four modes (tabs):

1. **Chat** — free-form text generation
2. **RAG** — retrieval-augmented generation over the demo knowledge base
3. **Extract** — structured extraction with the built-in `entities` schema
4. **Info** — system information (device, model backend, engine, tools)

Launch

```
cargo run -p graphdb-tui
```

Layout



Controls

Key	Action
Tab	Cycle through tabs (Chat → RAG → Extract → Info)
Enter	Submit the input
Ctrl+E	Toggle engine (llm ↔ needle)
Ctrl+Q / Esc	Quit

Modes

- **Chat:** Sends the input to `model.generate(...)`. The output is the generated text.
- **RAG:** Runs `RagPipeline::answer(...)` over the demo knowledge base. The output includes the answer and the source chunks.
- **Extract:** Runs `StructuredExtractor::extract(...)` with the built-in `entities` schema. The output is the extracted JSON.
- **Info:** Displays system information (device, model backend, engine, registered tools) — read-only, no input needed.

Model Selection

Default (Offline Stub)

By default, **every command** runs with an **offline stub model** (`StubTextModel`). This is a placeholder that:

- Generates boilerplate text (no real inference)
- Produces deterministic embeddings (no neural network)
- **Compiles and runs CPU-only with zero downloads**

Perfect for testing pipelines, CI, and examples.

What is `tokenizer.json` ?

A language model does not read text directly — it reads **token IDs** (integers). The **tokenizer** is the component that converts your text into those IDs before the model runs, and converts the model's output IDs back into text.

`tokenizer.json` is the **Hugging Face tokenizers format** — a single self-contained JSON file describing:

- the **vocabulary** (every token the model knows → its integer ID),
- the **merge rules** (for BPE-style tokenizers: how characters combine into tokens),
- the **normalization / pre-tokenization** rules (lowercasing, whitespace handling, byte-level encoding), and
- the **special tokens** (`<s>` , `</s>` , `<unk>` , padding, etc.).

Why it is a separate file from the model: a `.gguf` file holds the model **weights** (the learned numbers), while `tokenizer.json` holds the **text→ID mapping**. They must **match** — a model trained with one vocabulary will produce garbage if paired with a different tokenizer. Always download the `tokenizer.json` that ships with (or is referenced by) the specific model you are using.

Tip: If a model repo only provides `tokenizer.model` (SentencePiece) or split `vocab.json` + `merges.txt` , convert it to a single `tokenizer.json` with the Hugging Face `transformers` library:

```
python
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-3B-Instruct")
tok.save_pretrained("./out") # writes ./out/tokenizer.json
```

Real Local GGUF Model

To run a **real quantized Llama model**, pass `--model` and `--tokenizer` (or set them in `settings.json`):

```
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  generate "Explain Rust ownership"
```

The CLI will load the GGUF file via `candle-transformers` and run genuine inference. The `--model / --tokenizer` flags apply globally to **all** subcommands (including `agent` , `rag` , `ask` , etc.).

Supported model architectures (not just Llama)

The loader inspects the GGUF's `general.architecture` metadata field and dispatches to the matching `candle-transformers` parser automatically. You do **not** need to tell it which family a model is — just point `--model` at the `.gguf` . Supported architectures:

general.architecture	Parser used	Example models
llama , mistral , mixtral	quantized_llama	Llama 2/3, Mistral, Mixtral, TinyLlama
qwen2	quantized_qwen2	Qwen2 / Qwen2.5
qwen3	quantized_qwen3	Qwen3
gemma , gemma2 , gemma3	quantized_gemma3	Gemma / Gemma 2 / Gemma 3
phi3	quantized_phi3	Phi-3 / Phi-3.5
phi2 , phi	quantized_phi	Phi-2
glm4 , chatglm	quantized_glm4	GLM-4
(anything else)	falls back to quantized_llama	best-effort

Why this matters: earlier the loader always used the Llama parser, so loading a Qwen/Gemma/Phi GGUF failed with an error like:

```
failed to load model: from_gguf: cannot find llama.attention.head_count in metadata
```

That error means the GGUF was **not** a Llama-layout file — its metadata keys are prefixed with `qwen2.`, `gemma3.`, etc., not `llama.`. The architecture-aware dispatch now selects the correct parser, so those models load correctly. If you still hit `unsupported GGUF architecture "..."`, the model family isn't wired up yet — open an issue or use a Llama/Qwen/Gemma/Phi GGUF instead.

Check a file's architecture before downloading gigabytes:

```
python
from gguf import GGUFReader
r = GGUFReader("model.gguf")
print(r.fields["general.architecture"].parts[-1].tobytes().decode())
```

Where to Get Models

GGUF is the quantized model format used by `llama.cpp` and supported here via `candle`. To swap in a real model:

1. Pick a GGUF model from the [Hugging Face Hub](#) (search the `GGUF` library filter). Good small options for CPU:

- `bartowski/Llama-3.2-3B-Instruct-GGUF`
- `TheBloke/Llama-2-7B-Chat-GGUF`
- `Qwen/Qwen2.5-3B-Instruct-GGUF`

2. Download a quantization (a single `.gguf` file). `Q4_K_M` is a good speed/quality balance:

```

pip install -U "huggingface_hub[cli]"

# Download the weights (one .gguf file):
huggingface-cli download bartowski/Llama-3.2-3B-Instruct-GGUF \
  Llama-3.2-3B-Instruct-Q4_K_M.gguf \
  --local-dir ~/models

# Download the matching tokenizer.json (from the original, non-GGUF repo):
huggingface-cli download meta-llama/Llama-3.2-3B-Instruct \
  tokenizer.json --local-dir ~/models

```

3. Point the agent at them:

```

cargo run -p graphdb-cli -- \
  --model ~/models/Llama-3.2-3B-Instruct-Q4_K_M.gguf \
  --tokenizer ~/models/tokenizer.json \
  agent

```

...or set them once in `settings.json` so you never have to type the flags again.

Quantization cheat-sheet (smaller = faster + less RAM, but lower quality):

Suffix	Bits/weight	Use when
Q2_K / Q3_K_S	~2-3	Very tight RAM; noticeable quality loss
Q4_K_M	~4	Recommended default — best balance
Q5_K_M	~5	More quality, a bit slower
Q6_K / Q8_0	6-8	Near-full quality; needs more RAM

GPU Acceleration

GPU backends are **off by default** to keep builds fast and portable. To enable GPU acceleration, you must **rebuild** with the corresponding Cargo feature flag — just passing `--device cuda` at runtime won't work if the binary wasn't compiled with GPU support.

NVIDIA CUDA

```
# Build with CUDA support (one-time):
cargo build -p graphdb-cli --features cuda --release

# Then run with --device cuda:
./target/release/graphdb-cli \
  --device cuda \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  agent

# Or via cargo run (rebuilds if needed):
cargo run -p graphdb-cli --features cuda --release -- \
  --device cuda \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  agent
```

Requirements: CUDA toolkit 11.x or 12.x, cuDNN. Install via [NVIDIA's docs](#).

Apple Metal (M1/M2/M3)

```
# Build with Metal support (one-time):
cargo build -p graphdb-cli --features metal --release

# Then run with --device metal:
./target/release/graphdb-cli \
  --device metal \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  agent
```

Requirements: macOS 12.3+ with Apple Silicon. No additional setup needed.

Available Features

Feature	Backend	Notes
cuda	NVIDIA GPU	via cuDNN; requires CUDA toolkit
metal	Apple GPU	M1/M2/M3; macOS only
rocm	AMD GPU	experimental; Linux only
mkl	Intel MKL	CPU acceleration for x86
flash-attn	Flash Attention 2	CUDA only; faster attention

Tip: You can set "device": "cuda" in [settings.json](#) so you never have to type the flag again — but remember the binary must still be compiled with `--features cuda`.

Configuration (settings.json)

The `agent` REPL is **configured without recompiling** through a `settings.json` file. This is the recommended way to set a model, device, generation parameters, skills folder, and startup context — so you don't have to retype CLI flags every time.

A ready-to-copy template ships in the repo: `graphdb-agent.settings.example.json`.

```
# Copy the template and edit it:
cp graphdb-agent.settings.example.json graphdb-agent.settings.json
$EDITOR graphdb-agent.settings.json

# The agent auto-discovers ./graphdb-agent.settings.json:
cargo run -p graphdb-cli -- agent

# Or point at any file explicitly:
cargo run -p graphdb-cli -- --settings ~/my-agent.json agent
```

Inside the REPL, type `/settings` to see the effective configuration and where it was loaded from.

Discovery Order & Precedence

The agent looks for a settings file in this order (first match wins):

1. `--settings <path>` — explicit flag
2. `$GRAPHDB_AGENT_SETTINGS` — environment variable
3. `./graphdb-agent.settings.json` — current directory
4. `~/.config/graphdb/agent.settings.json` — user config

If none exist, built-in defaults apply (offline stub model, CPU).

Every value can still be overridden on the command line. The precedence is:

```
CLI flag > settings.json > built-in default
```

For example, `--device cpu` on the command line wins over `"device": "cuda"` in the file. This lets you keep a stable config file and override just one thing for a single run.

Full Reference

```
{
  "model": {
    "path": "~/models/llama-3.2-3b-instruct-q4_k_m.gguf",
    "tokenizer": "~/models/tokenizer.json",
    "device": "cpu"
  },
  "generation": {
    "max_tokens": 512,
    "temperature": 0.2
  },
  "skills_dir": "./my-skills",
  "context_dir": "./src",
  "history_file": "~/.graphdb_agent_history"
}
```

Key	Type	Default	Description
<code>model.path</code>	string null	<code>null</code>	Path to a GGUF model file. <code>null</code> → offline stub model.
<code>model.tokenizer</code>	string null	<code>null</code>	Path to the matching <code>tokenizer.json</code> . Required when <code>model.path</code> is set.
<code>model.device</code>	string	<code>"cpu"</code>	Compute device: <code>"cpu"</code> , <code>"cuda"</code> , or <code>"metal"</code> .
<code>generation.max_tokens</code>	integer	256	Maximum new tokens per answer.
<code>generation.temperature</code>	number	<code>0.0</code>	Sampling temperature. <code>0.0</code> = greedy / deterministic.
<code>skills_dir</code>	string null	<code>null</code>	Folder of <code>*/SKILL.md</code> skills. <code>null</code> → built-in <code>model-hub/examples/skills</code> .
<code>context_dir</code>	string null	<code>null</code>	Directory whose <code>.rs</code> / <code>.md</code> / <code>.txt</code> files are ingested at startup.
<code>history_file</code>	string null	<code>~/.graph-db_agent_history</code>	Persistent readline history file.

Notes:

- Paths beginning with `~/` are expanded to your home directory.
- Any field may be omitted — missing fields fall back to their default.
- Unknown keys are **rejected** with a helpful error listing valid keys (catches typos early).

Common Recipes

Always use my local model (no flags needed):

```
{
  "model": {
    "path": "~/models/Llama-3.2-3B-Instruct-Q4_K_M.gguf",
    "tokenizer": "~/models/tokenizer.json",
    "device": "cpu"
  }
}
```

Longer, more creative answers:

```
{ "generation": { "max_tokens": 1024, "temperature": 0.7 } }
```

Work on a specific project with custom skills:

```
{
  "context_dir": "~/projects/my-app/src",
  "skills_dir": "~/projects/my-app/.graphdb-skills"
}
```

Customizing & Extending

The agent has three extension points, from easiest to most involved:

Add a...	Requires code?	Where
Skill	No — just a <code>SKILL.md</code> file	<code>skills_dir</code> folder
Tool	Yes — Rust (schema + executor)	<code>model-hub/src/pipeline/tools.rs</code>
Trained Needle model	No Rust — Python + JSONL data	<code>python/</code> (cactus-needle)

Adding a New Skill

Skills need **no code and no rebuild** — they are Markdown files discovered at startup. This is the easiest way to steer the agent's behaviour.

1. Create a folder + `SKILL.md` in your `skills_dir` (default `model-hub/examples/skills/`):

```
mkdir -p model-hub/examples/skills/test-writer
$EDITOR model-hub/examples/skills/test-writer/SKILL.md
```

2. Write the skill with `---` frontmatter followed by the instruction body:

```

---
name: test-writer
description: Write unit tests for a piece of code.
cues: test, unit test, write tests, coverage, assert, test case
---

```

When asked to write tests, produce a compact test module.

- Ground every test in the retrieved code (real function names and signatures).
- Cover the happy path plus at least one edge case (empty input, error path).
- Use the project's existing test conventions if visible in the context.
- Output only the test code in a fenced block, no prose padding.

3. Restart the agent and confirm it loaded:

```
◆ /skills
```

How selection works: for each task, the agent lowercases the input and counts how many of a skill's `cues` appear in it. The highest-scoring skill wins and its instruction body is injected into the LLM prompt (for Moderate/Complex tasks). If no cue matches, no skill is injected.

Frontmatter fields:

Field	Required	Description
<code>name</code>	Yes	Unique skill name (shown in <code>/skills</code> , <code>+ skill</code> trace).
<code>description</code>	Yes	One-line summary (shown in <code>/skills</code>).
<code>cues</code>	Yes	Comma-separated trigger phrases (lowercased, substring-matched).
(body)	Yes	Everything after the closing <code>---</code> ; injected into the LLM prompt.

Tip: Make cues specific. Generic cues like `"code"` fire on almost everything and will crowd out more precise skills.

Adding a New Tool

Tools are the actions the agent can take against the knowledge base (`search_knowledge_base`, `list_documents`, `get_document_chunks`). Adding one requires **three edits in Rust**, all in `model-hub/src/pipeline/tools.rs`.

1. Declare the schema in `graphdb_tool_schemas()`:

```
ToolSchema::new(
  "count_documents",
  "Count how many documents are in the knowledge base.",
)
// .with_param("name", "type", "description", required)
.with_param("doc_type", "string", "restrict the count to a document type", false),
```

2. Implement execution in `ToolExecutor::execute` — add a match arm that runs against the `GraphBackend` and returns JSON:

```
"count_documents" => {
  let docs = graph.list_documents().await?;
  let doc_type = call.arguments.get("doc_type").and_then(|d| d.as_str());
  let count = match doc_type {
    Some(t) => docs.iter().filter(|d| d.doc_type == t).count(),
    None => docs.len(),
  };
  Ok(serde_json::json!({ "count": count }))
}
```

3. (Optional) Teach the Needle router in `model-hub/src/needle/agent.rs` → `detect_tool()` so the deterministic engine can pick it without an LLM:

```
// count intents → count_documents
if has("count_documents") && (q.contains("how many") || q.contains("count")) {
  return Some(ToolCall {
    name: "count_documents".to_string(),
    arguments: json!({}),
  });
}
```

4. Rebuild and verify:

```
cargo build -p graphdb-cli
cargo run -p graphdb-cli -- agent
# ♦ /tools ← your new tool should appear
```

What the engines do with a tool:

- **LLM routing** (Complex tasks) automatically sees the new schema — it's serialized into the router prompt via `to_function_json()`, so no extra work is needed beyond step 1-2.
- **Needle routing** (Simple/Moderate tasks) only picks tools it has heuristics for — hence the optional step 3.

The Rust `ToolSchema` list intentionally mirrors the Python `GRAPHDB_TOOL_SCHEMAS` in `python/ai_memory/needle_agent.py`. If you want the same tool available to trained Needle models, add a matching entry there too.

Training a Needle Model for a Task

The **Needle engine** is deterministic (regex/heuristics) out of the box. For higher accuracy on your documents and phrasings, you can **train a compact Needle2 function-calling model** (via the `cactus-needle` library) on data generated from your own knowledge base. This is a **Python** workflow — no Rust required.

Needle2 is a small **embedded function-calling model** (not a chat LLM). Every response is a structured JSON tool call, or `[]` for off-topic queries. It is fine-tuned with LoRA adapters on JSONL data and exported to a `.cact` archive.

1. Install the training extras:

```
pip install 'cactus-needle[train]'          # CPU
pip install 'cactus-needle[train,gpu]'      # NVIDIA GPU
pip install 'cactus-needle[train,metal]'    # Apple Silicon
```

2. Ingest your documents into a knowledge group and export training data (Python):

```
from ai_memory.needle_agent import NeedleAgentGroup
from ai_memory.embedder import LocalEmbedder
from ai_memory.document_loader import DocumentLoader
from graphdb.store import GraphStore

group = NeedleAgentGroup(
    name="my_docs",
    store=GraphStore(),
    embedder=LocalEmbedder(),
    # small chunks → more positive examples to interleave off-topic ones
    loader=DocumentLoader(chunk_size=200, min_chunk_len=20),
    system="knowledge_group: my_docs; domain: my-project",
)

group.ingest(["docs/guide.md", "src/main.rs"])  # your files
path = group.export_training_data("my_train.jsonl", k=200)
print("wrote", path)
```

This produces **Needle-format JSONL**: each line has `tools`, `answers`, and `reasoning` fields. A configurable fraction (default **1-in-8**) are off-topic examples with `answers: []`, which teach the model **not** to call a tool on every query.

3. Fine-tune and build the `.cact` archive (terminal):

```
needle finetune my_train.jsonl --epochs 20 --out my_adapter.pkl
needle build checkpoints/needle2.pkl --lora my_adapter.pkl --out my.cact
```

4. Load the trained weights and run live inference (Python):

```
group.load_weights("my.cact")
result = group.run("How does vector search work?")
print(result["type"], result["function_calls"], result["confidence"])
```

5. (Optional) Route across multiple trained groups with `NeedleOrchestrator` — each group specializes in its own document domain, and the orchestrator picks the best-matching group per query:

```
from ai_memory.needle_agent import NeedleOrchestrator

orch = NeedleOrchestrator(groups=[tech_group, research_group], embedder=LocalEmbedder())
chosen = orch.route("What is LoRA?")  # → research_group
```

Data-quality guidance (from the shipped example): tool selection improves with a few hundred clean examples; argument grounding needs thousands of varied phrasings; keep ~1-in-8 off-topic examples so the model learns to stay silent when appropriate.

See the full runnable walkthrough in [python/examples/example_needle_agent.py](#) and the API reference in [python/SDK.md](#) (Needle2 Integration section).

Note: the trained `.cact` model is currently consumed by the **Python** Needle integration. The Rust agent REPL's Needle engine uses the built-in heuristics; wiring a trained `.cact` into the Rust REPL is not yet supported.

Engine Selection (LLM vs. Needle)

Many pipelines support **two engines** for tool calling, extraction, and embeddings:

Engine	Description	Speed	Accuracy
<code>needle</code>	Heuristic, regex-based, fully offline (no weights)	Instant	Good for deterministic tasks (list, lookup, simple entities)
<code>llm</code>	Neural text model (stub or real GGUF Llama)	Slower	Better for ambiguous queries, reasoning, complex extraction

Where to set it:

Command	Flag	Default
<code>embed</code>	<code>--engine <ENGINE></code>	<code>needle</code>
<code>rag</code>	<code>--engine <ENGINE></code>	<code>needle</code>
<code>extract</code>	<code>--engine <ENGINE></code>	<code>needle</code>
<code>ask</code>	<code>--engine <ENGINE></code>	<code>needle</code>
<code>agent</code>	(automatic routing by difficulty)	N/A

Example:

```
# Use Needle for fast deterministic extraction:
cargo run -p graphdb-cli -- extract "Alice at OpenAI" --engine needle

# Use the LLM for more nuanced extraction:
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  extract "Alice at OpenAI" --engine llm
```

In the **agent REPL**, engine selection is **automatic** — the difficulty router picks the cheapest engine that can handle the task:

- **Simple** → Needle only (no LLM)
- **Moderate** → Needle routes, LLM composes
- **Complex** → LLM routes + composes

You can toggle the default engine in the **TUI** by pressing **Ctrl+E**.

Python SDK

For Python users, the `ai_memory` package provides:

- `GraphStore` — the core graph storage (PyO3 wrapper around the Rust engine)
- `DatasetBuilder` — ingest documents and generate training datasets
- `NeedleAgentGroup` + `NeedleOrchestrator` — embedded function-calling agents (Needle2 integration)
- `PyCommunityDetector`, `PyGraphRAGRetriever`, `PyJEPAGraphRAG` — community detection, GraphRAG, and the JEPA world model

See [python/SDK.md](#) for the full API reference and [python/README.md](#) for setup instructions.

Further Reading

- [EXAMPLES.md](#) — Runnable examples for each capability (GraphBackend, JEPA, OCR, image generation, transcription, web scraping, segmentation, Claude Code agent)
- [README.md](#) — Project overview, architecture, and build instructions
- [python/SDK.md](#) — Python API reference
- [model-hub/examples/](#) — Example source code (`.rs` files)
- [model-hub/examples/skills/](#) — Skill definitions (`.md` files)
- [graphdb-agent.settings.example.json](#) — Copy-ready `settings.json` template
- [python/examples/example_needle_agent.py](#) — Needle model training walkthrough

Tip: Run any command with `--help` to see its full usage:

```
cargo run -p graphdb-cli -- --help
cargo run -p graphdb-cli -- generate --help
cargo run -p graphdb-cli -- agent --help
```